

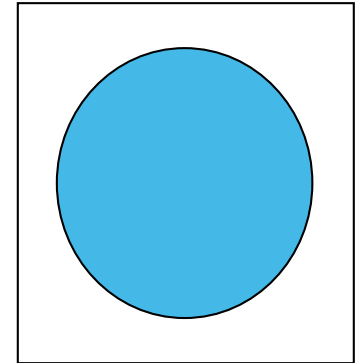
Lab 1

K Nearest Neighbors, Decision trees

Problem

distinguish the points belonging to:

1. circle of radius 1, centered in 0,0
2. square with sides of length $L=2.5$, centered in (0,0) from which region 1 has been removed.

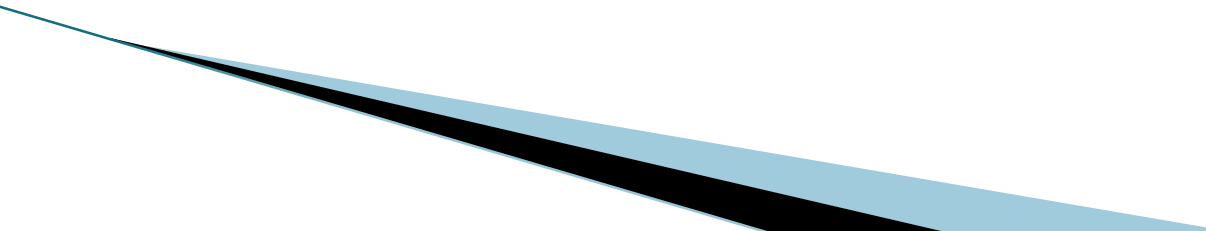


Classify the data in the following files:

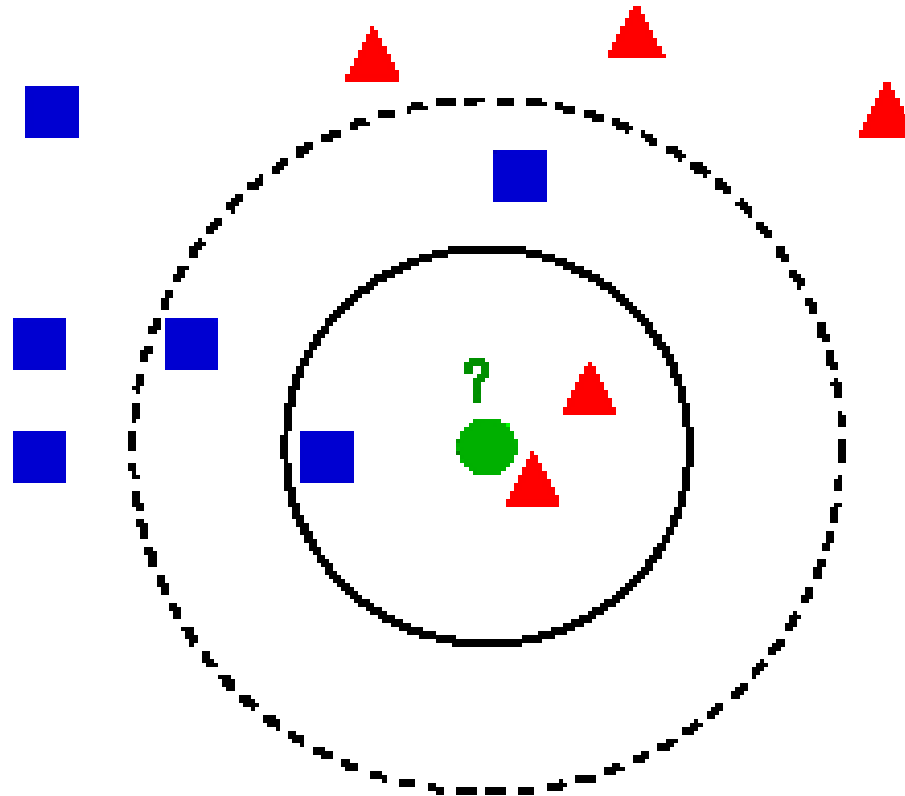
<code>circletrain.arff</code>	training set (100 samples)
<code>circletest.arff</code>	test set (100 different samples)
<code>circleall.arff</code>	test set with the whole domain sampled at a small and constant step (can be taken as ground truth)

Exercise 1 (k Nearest Neighbors)

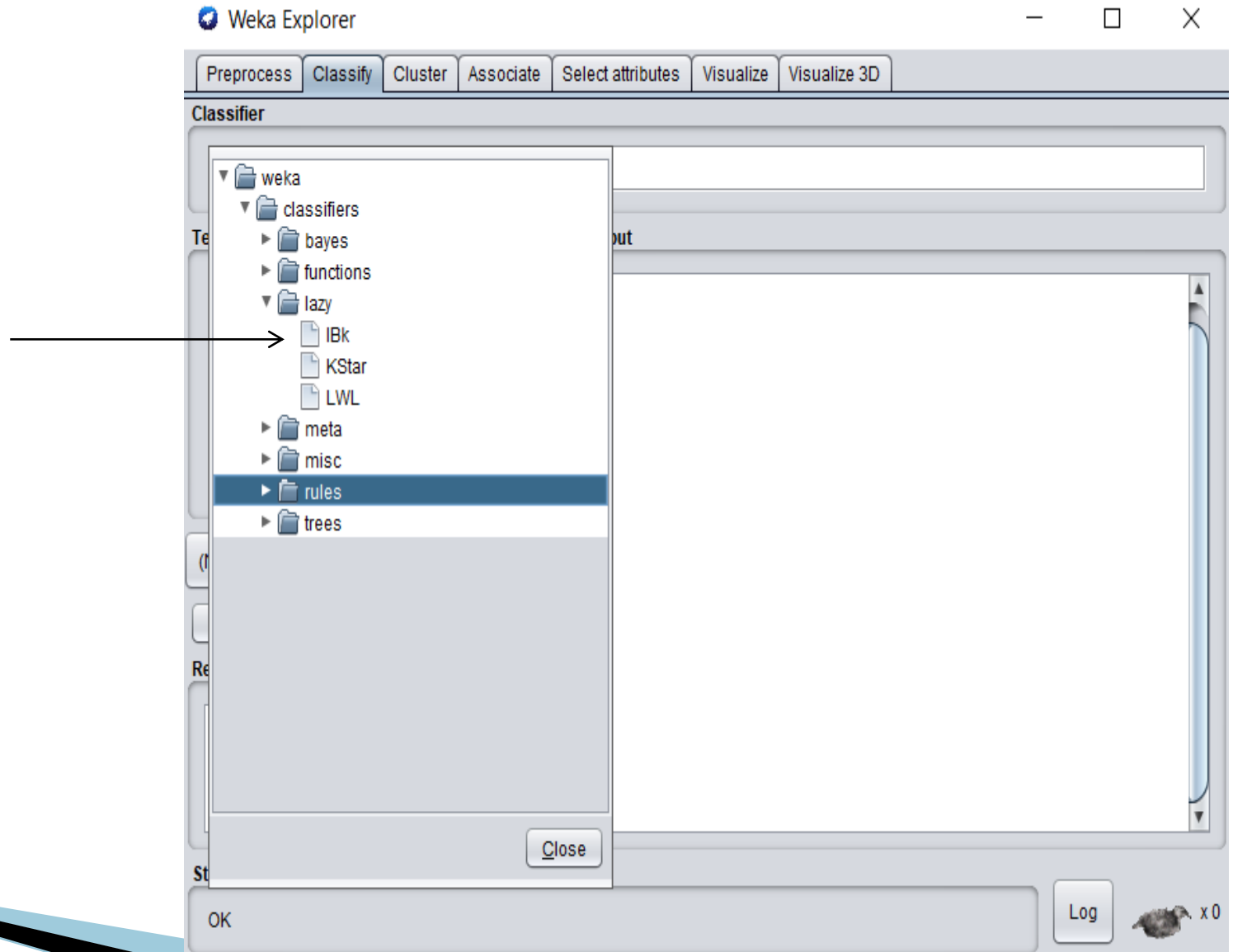
A kNN (k Nearest Neighbors) classifier assigns a label to a sample X according to the labels of the k (k odd) closest samples in the training set (**majority vote**).

1. Read X
 2. Find the k samples closest to X in the training set according to a pre-defined distance measure (usually the Euclidean distance).
 3. Assign to X the label that is most frequently associated to the k examples of the training set found in the previous step.
- 

k Nearest Neighbors



k Nearest Neighbors in WEKA (Ibk)



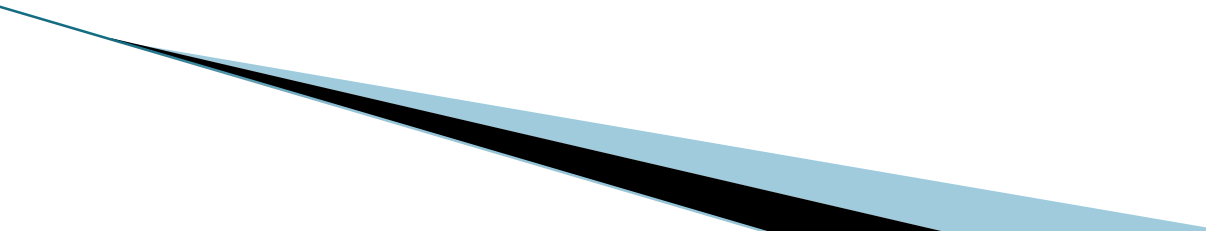
Exercise 1 (k Nearest Neighbors)

- ▶ Classify the test set in `circletest.arff` using the kNN algorithm based on the samples in the training set `circletrain.arff`.
- ▶ Use $k=1,3,5, \dots$ and observe how the accuracy obtained on `circletest.arff` changes.
- ▶ Compare the results with those obtained by the user-defined classifier
- ▶ Using `circletest.arff` as a validation set, how would you choose the best k ?
- ▶ Verify the results on `circleall.arff` (a dense uniform sampling of the whole domain)

Exercise 1 b (representation)

- ▶ Suppose the inner region is another square (centered as the circle, side length $l=1.76$)
- ▶ “Correct” the labels of the training and test set accordingly (create a new .arff file with the same points and the correct labels for this new problem)
- ▶ Would you expect to obtain better or worse accuracy when running J48 on the new data?
- ▶ Why? (Hint: try writing the solution to the problem as a set of ANDed equations in x and y which define the inner region)
- ▶ Run J48 and check!

Exercise 1 b (representation)

- ▶ Consider a representation in which the two variables are $z=x^2$ and $t=y^2$
 - ▶ Build new datasets using the same points but this new representation
 - ▶ Would you expect to obtain better or worse accuracy when running J48 on the new data?
 - ▶ Why?
 - ▶ Run J48 and check!
- 

Exercise 1 b (representation)

- ▶ Define a new variable as a function of x and y such that you can obtain 100% accuracy with the smallest possible tree.
- ▶ Build new datasets using the same points but this new representation.
- ▶ Run J48 and see if it is able to find a solution having just one node.

(hint for your report: test the classifier on the data in `circleall.arff` and use the function “Visualize classifier errors” accessible by right-clicking on the results list)

Exercise 2

- ▶ Open WEKA Explorer
- ▶ Load the file `Bigtest1_104.arff`

It contains patterns representing small (resol. 13x8) binary images of digits taken from license plates.

For instance (patterns are described row-wise)

```
0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0,  
1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0,  
0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1,  
1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0
```

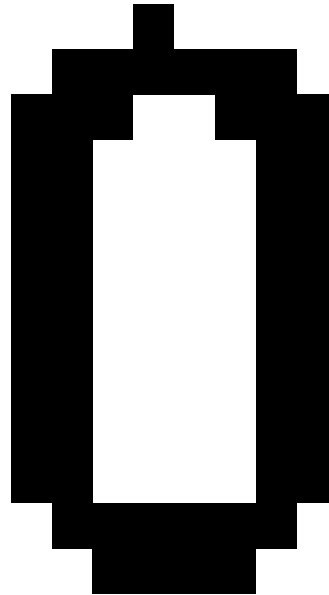
Exercise 2

The pattern

0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0,
0, 0, 1, 1, 1, 1, 0, 0, 0

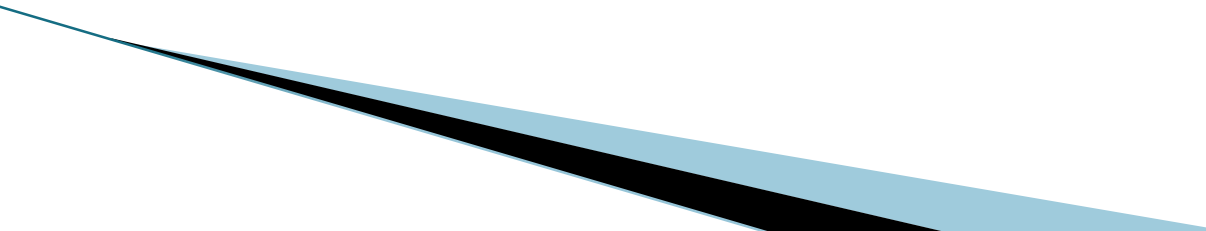
corresponds to

00010000
01111110
11100111
11000011
11000011
11000011
11000011
11000011
11000011
11000011
11000011
01111110
00111100

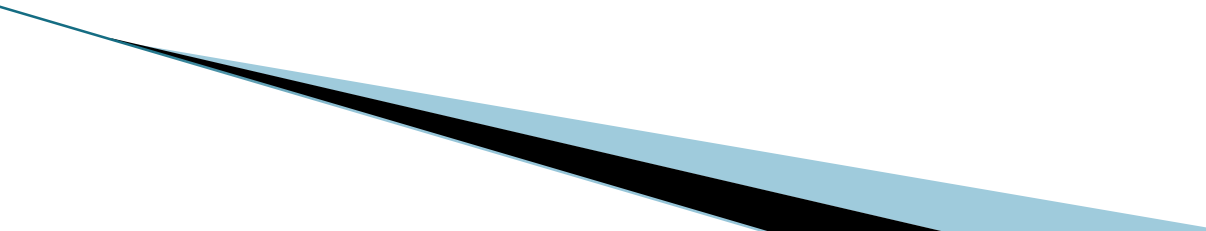


0 (label)

Exercise 2

- ▶ Open WEKA Explorer
 - ▶ Load the file `Bigtest1_104.arff`
 - ▶ Select `random split 66%` as test modality
 - ▶ Select J48 as a classifier and run it using its default parameters.
 - ▶ Observe the results and the structure of the tree that has been generated
- 

Exercise 2

- ▶ Modify the seed for the random selection of the training and test sets (More Options -> Random seed) and create a new tree. Leave all other settings unchanged. Repeat this step 5 (or more) times and observe how the sample distribution and the test results change.
 - ▶ Compute the average accuracy obtained on the 5 training sets
 - ▶ Load `Bigtest2_104.arff` as a test file (Supplied test set)
 - ▶ Run J48 again
 - ▶ Compare the results obtained to the average of the previously performed runs.
- 

Exercise 2b (overfitting control)

- ▶ Keep using `Bigtest2_104.arff` as test file
- ▶ Run a set of training sessions keeping the default parameters but increasing M (minimum number of training samples in a leaf) in each session ($M=1,2,3 \dots$)
- ▶ Compare the results obtained using the different values for M with the results obtained testing the tree on the whole training set `Bigtest1_104.arff`

(Use training set as test modality).

- ▶ Compare what happens changing M and testing the classifiers on the training set. Justify the different behavior that can be observed.
- ▶ Take note of the value of M corresponding to which J48 obtains the best accuracy on `Bigtest2_104.arff`

Exercise 2b (overfitting control)

Notes

- ▶ Setting $M=1$, it would be theoretically possible to learn a tree which ‘memorizes’ each sample in the training set obtaining 100% accuracy on the training set.
- ▶ This does not happen because:
 - By choosing the attribute which yields the best information gain the algorithm performs a ‘greedy search’ which may not lead to the optimal choice in the long run.
 - Some ‘tricks’ (like, for instance, pruning) are set by default in J48
 - The number of attributes may not be large enough

Exercise 2b (overfitting control)

Notes

- ▶ When a parameter of a classifier controls the generalization capacity (here, the parameter M), one can start from a value which largely underfits the training data – i.e., the classifier is not able to distinguish properly the different classes (here, the larger M , the less the classifier can fit data) – then change it to increase the fitting capacity (here, decrease M).
- ▶ For a while, the classifier performance will improve both when tested on the training and when tested on a validation set, up to a certain value, where the performance on the training set will still improve, while those on the validation set will start worsening.
- ▶ Such a value is the optimal value for the parameter.

Scikit-Learn: Machine learning in Python

Mattia Pellegrino, Ph.D Fellow
mattia.pellegrino@unipr.it

WHAT

- **Simple** and **efficient** tools for data **mining** and data **analysis**
- Accessible to everybody, and reusable in various contexts
- Built on **NumPy**, **Scipy**, and **matplotlib**
- Open source, commercially usable
- Good Documentation, Good Community support

WHY

Sickit Provides

- **Classification**
 - Identifying to which category an object belong to
 - **Applications:** Spam detection, Image recognition
 - **Algorithms:** SVC, nearest neighbors, random forest, etc.
- **Regression**
 - Predicting a continuous-valued attribute associated with an object
 - **Applications:** Drug response, stock prices
 - **Algorithms:** SVR, ridge regression, etc.
- **Clustering**
 - Automatic grouping of similar objects into sets
 - **Applications:** Customer segmentation, grouping experiment outcomes
 - **Algorithms:** k-Means, spectral clustering, etc.

WHY

Sickit Provides

- **Other tasks available:**
 - **Dimensionality reduction**
 - Reducing the number of random variables to consider
 - **Model selection**
 - Comparing, validating and choosing parameters and models
 - **Preprocessing**
 - Feature extraction and normalization

HOW?

Installation

- **Scikit-learn requires:**

- Python
- NumPy
- SciPy
- joblib
- threadpoolctl

Installation

- **Scikit-learn install:**

- `Python -m pip install -U pip`
- `Pip install scikit-learn`
- `Pip install numpy`
- `Pip install scipy`
- `Pip install matplotlib`

Machine Learning



what society thinks I
do



what my friends think
I do



what my parents think
I do

$$L_y = \gamma \|\mathbf{w}\|^2 - \sum_{i=1}^n a_i y_i (\mathbf{x}_i \cdot \mathbf{n} + b) + \sum_{i=1}^n a_i$$

$$a_i \geq 0, \forall i$$

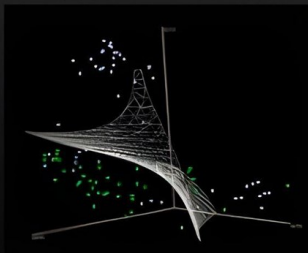
$$\mathbf{n} \cdot \sum_{i=1}^n a_i y_i \mathbf{x}_i, \sum_{i=1}^n a_i y_i = 0$$

$$\nabla \hat{J}(\theta_t) = \frac{1}{n} \sum_{i=1}^n \nabla \ell(x_{i(t)}, y_{i(t)}; \theta_t) + \nabla r(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta_t \nabla \ell(x_{i(t)}, y_{i(t)}; \theta_t) - \eta_t \cdot \nabla r(\theta_t)$$

$$\mathbb{E}_{i(t)}[\ell(x_{i(t)}, y_{i(t)}; \theta_t)] = \frac{1}{n} \sum_i \ell(x_i, y_i; \theta_t)$$

what other programmers
think I do



what I think I do

```
>>> from sklearn import svm
```

what I really do

Esempio di pipeline di classificazione (1/3)

```
# ml_pipeline_example.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# === 1. Load data from CSV ===
# Assume your CSV file has columns: feature1, feature2, ..., featureN, target
data = pd.read_csv("data.csv")

# Separate features (X) and target (y)
X = data.drop("target", axis=1)
y = data["target"]
```

Esempio di pipeline di classificazione (2/3)

```
# === 2. Split into train, validation, and test sets ===
```

```
# First split into train (70%) and temp (30%)
```

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42,  
stratify=y)
```

```
# Split temp into validation (15%) and test (15%)
```

```
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42,  
stratify=y_temp)
```

```
print(f"Training set: {X_train.shape}")
```

```
print(f"Validation set: {X_val.shape}")
```

```
print(f"Test set: {X_test.shape}")
```

```
# === 3. Train a Decision Tree model ===
```

```
clf = DecisionTreeClassifier(random_state=42)
```

```
clf.fit(X_train, y_train)
```


Esempio di pipeline di classificazione (3/3)

```
# === 4. Evaluate on validation set (optional tuning step) ===
```

```
val_pred = clf.predict(X_val)
```

```
val_acc = accuracy_score(y_val, val_pred)
```

```
print("\nValidation Accuracy:", val_acc)
```

```
# === 5. Evaluate on test set ===
```

```
test_pred = clf.predict(X_test)
```

```
print("\n=== Test Set Evaluation ===")
```

```
print("Accuracy:", accuracy_score(y_test, test_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, test_pred))
```

```
print("\nConfusion Matrix:\n", confusion_matrix(y_test, test_pred))
```

EXERCISE – CLASSIFY IRIS FLOWERS

- **Understanding the Problem**

- The Iris dataset contains 150 samples of 3 different species of iris flowers: Setosa, Versicolour, and Virginica.
- Each flower is described by 4 features: sepal length, sepal width, petal length, and petal width.
- The goal is to build a classification model that identifies the species of iris flowers based on the features.

- **Algorithms to Use**

- **Use** DecisionTreeClassifier as the model

EXERCISE – CLASSIFY IRIS FLOWERS

- **Plan for Solution**

- Load the **Iris dataset**
- Implement a kNN **Classifier**
- Implement a **Decision Tree Classifier**
- Compare their performances using accuracy and confusion matrix

Lab 2

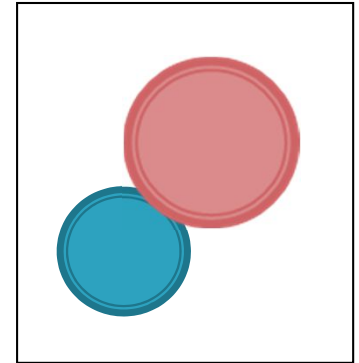
Clustering



Problem

Distinguish the points belonging to:

1. Gaussian distribution centered in $(3,3)$
2. Gaussian distribution centered in $(7,7)$



Cluster the data in the following arff files:

gausstrain.arff training set (100 samples)

gausstest.arff test set (100 more samples)

generated using MATLAB, function `gaussgen(N,m,v)`

N = n. of samples m = means (2D row vector)

v = variances (2D row vector)

Notes: Variances are equal along both axes

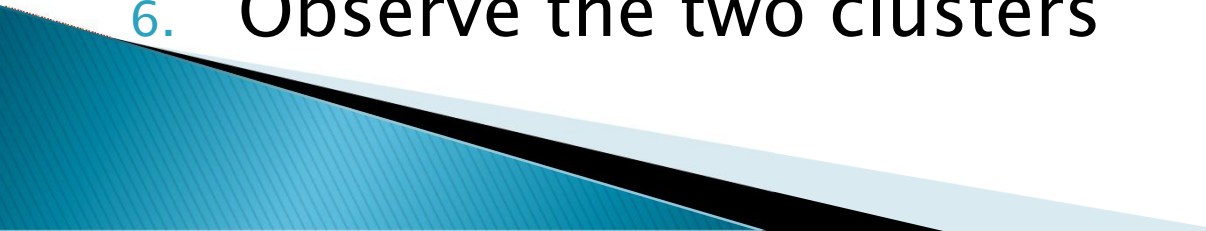
Similar functions exist virtually in any language

Exercise 0 (basic K-Means clustering)

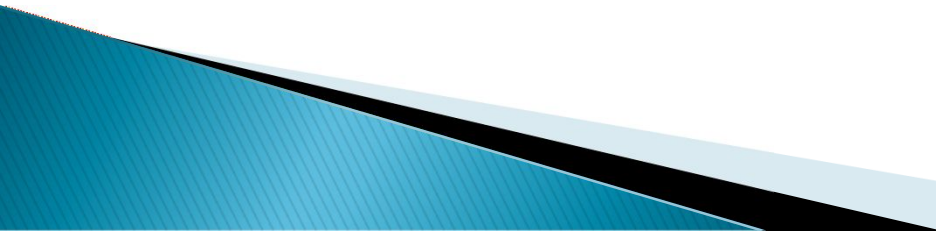
Weka provides K-Means clustering in the package SimpleKMeans

1. Open gausstrain.arff
 2. In the Cluster Tab, select SimpleKMeans as clustering algorithm
 3. Select “Use training set” as “Cluster mode”
 4. Select “Ignore attributes” to ignore the class attribute “cluster”
 5. Set 2 as the number of desired clusters
- 

Exercise 0 (basic K-Means clustering)

1. Run K-Means using the default parameters
 2. Observe where K-Means has placed the centroids for the two distributions. How far are they from the Gaussian centers?
 3. Right click on the result buffer and select “Visualize cluster assignments”
 4. Patterns are assigned to the cluster represented by the closest centroid
 5. Select attribute x as X and attribute y as Y
 6. Observe the two clusters
- 

Exercise 0b (supervised classification)

1. Open gausstrain.arff
 2. In the Cluster Tab, select SimpleKMeans as clustering algorithm
 3. Select “Classes to Clusters evaluation” as “Cluster mode” using “cluster” as class attribute
 4. Centroids/Clusters are assigned to the class to which the majority of patterns in the cluster belong
 5. Set 2 as the number of desired clusters
- 

Exercise 0b (supervised classification)

1. Run K-Means using the default parameters
 2. Observe where K-Means has placed the centroids for the two distributions. How far are they from the Gaussian centers?
 3. Right click on the result buffer and select “Visualize cluster assignments”
 4. Select attribute x as X and attribute y as Y
 5. Observe the two clusters and where samples that have been misclassified are located.
- 

Exercise 0c (data with higher variance)

1. Open gausstrainhv.arff
 2. Open the “Visualize” Tab
 3. Select attribute x as X and attribute y as Y
 4. Observe the new data: Gaussian means are the same but variance has increased by 50%
 5. Try to predict which points belonging to the two distributions will be assigned to the “wrong” cluster (the one that includes mainly points from the other distribution)
 6. Discussion: is it really the “wrong” cluster ?
- 

Exercise 0c (data with higher variance)

1. Run K-Means
 2. Observe where K-Means has placed the centroids for the two distributions. How far are they from the Gaussian centers?
 3. Right click on the result buffer and select “Visualize cluster assignments”
 4. Select attribute x as X and attribute y as Y
 5. Observe the two clusters and where samples that have been misclassified are located.
- 

Exercise 1

- ▶ Load the file `Bigtest1_104.arff`

It contains patterns representing small (res. 13x8) binary images of digits taken from license plates.

For instance (patterns are described row-wise)

```
0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0,  
1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0,  
0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1,  
1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0
```

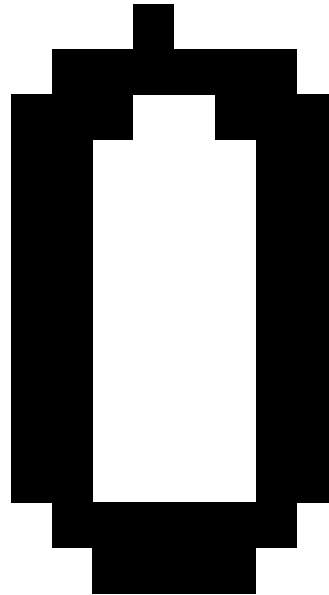
Exercise 1

The pattern

0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1,
1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0,
0, 0, 1, 1, 1, 1, 0, 0, 0

corresponds to

00010000
01111110
11100111
11000011
11000011
11000011
11000011
11000011
11000011
11000011
11000011
01111110
00111100



0 (label)

Exercise 1

1. In the Cluster Tab, select SimpleKMeans as clustering algorithm
 2. Select “Use training set” as “Cluster mode”
 3. Select “Ignore attributes” to ignore the class attribute “class”
 4. Set 10 as the number of desired clusters
 5. Observe the cluster assignments
- 

Exercise 1

1. Run K-Means
2. Observe where K-Means has placed the centroids.
3. Write your own program to visualize the centroids as greyscale images (remember that the centroids are 104-dimensional vectors and images 13x8 matrices) or use MATLAB function `plotpattern(pat1, pat2)` to visualize up to two centroids at one time.
4. They do look like the figures they represent: does this surprise you? Why?

[The two data sets in MATLAB format are in the file `bigtest.mat`]

Exercise 1

Notes

1. The grey level of each pixel of the “centroid pattern” represents an estimation of the probability that that pixel is ON when the pattern belongs to the cluster
 2. In fact, since the centroid is the average of the patterns belonging to one cluster, the grey level of each pixel is a value between 0 and 1, proportional to the frequency with which the pixel is ON within that cluster.
- 

Exercise 2

1. In the Cluster Tab, select XMeans as the clustering algorithm
 2. Select “Classes to clusters evaluation” as “Cluster mode”, specifying “class” as the class attribute
 3. Set 10–40 (min–max) as the number of desired clusters and default values for all other parameters.
 4. Observe the cluster assignments and how many clusters have been found. Notice that the algorithm will only assign one centroid to each class: in particular, it is the centroid whose cluster includes the largest number of samples of the class under consideration.
- 

Exercise 2

1. Considering that only one cluster is automatically associated to each class, assign all unlabeled clusters to the class which is most frequently represented within it (the class corresponding to the largest value in each column of the “class to clusters matrix”) and compute the actual confusion matrix.

Notice that the actual number of correctly classified patterns (and therefore accuracy) can be estimated by subtracting from the number of errors the sum of the largest values of the columns of the “classes to cluster” matrix corresponding to unlabeled clusters.

Exercise 2b

1. Run Xmeans several times changing the max number of desired centroids until the number of centroids which are actually found does not change any more (if this ever happens).
2. For each run, use the data in the test set `Bigtest2.arff` to validate the classifier (you may need to write your own code to assign test samples to clusters/classes and compute the accuracy)
3. What can you say about overfitting after analyzing the results ?



**UNIVERSITÀ
DI PARMA**

**MASTER DEGREE IN COMPUTER ENGINEERING
MACHINE LEARNING COURSE**

Contents

1. Introduction to PyTorch.
2. Tensors.
3. Datasets, DataLoaders, and Transforms.
4. MLP, Loss Functions, and Optimizers.
5. Training Loop & Eval Loop.

PyTorch: a flexible framework for deep learning

What is PyTorch?

- The most widely used open-source Deep-Learning framework.
- Provides automatic differentiation and GPU acceleration for efficient computation.
- Built around **tensors**.

Enables easy definition of:

- **Datasets** and data preprocessing,
- **Models**,
- **Loss functions** and **optimizers**.





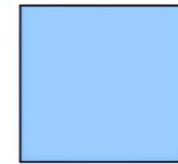
Tensors: the core data structure in PyTorch

What is a **tensor**?

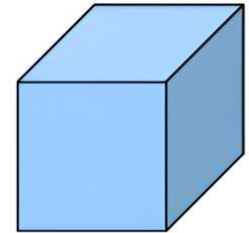
- **Multi-dimensional** array.
- Supports **automatic differentiation** for gradient computation.
- Enables fast computation on **GPU**.
- Optimized for **mathematical operations**.



1d-tensor



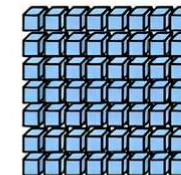
2d-tensor



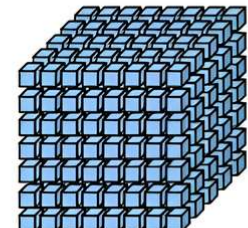
3d-tensor



4d-tensor



5d-tensor



6d-tensor



Tensors: the core data structure in PyTorch

How can we create tensors?

- From Python lists: `torch.Tensor([[1, 2], [3, 4]])`
- With fixed values: `torch.zeros(2, 3)`
- With random values: `torch.rand(2, 3)`

What operations can we do with tensors?

- Move to GPU: `t.to('cuda')`
- Indexing and slicing: `t[2]`, `t[:,0]`
- Element-wise ops: `x+y`, `x*y`
- Matrix multiplication: `torch.matmul(x, y)`



Tensors: the core data structure in PyTorch

Why tensors matter for Deep Learning?

- Mostly everything can be seen as a tensor.
- For example, RGB images are 3D tensors.





Datasets and DataLoaders in PyTorch

How can we handle datasets in PyTorch?

- **Datasets**: abstracts a collection of data samples (inputs and labels).
- **DataLoader**: iterator over a Dataset, supports batching, shuffling, and parallel loading.
- **Datasets** → data representation
- **DataLoader** → how we feed data to the model

```
from torch.utils.data import DataLoader  
train_loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

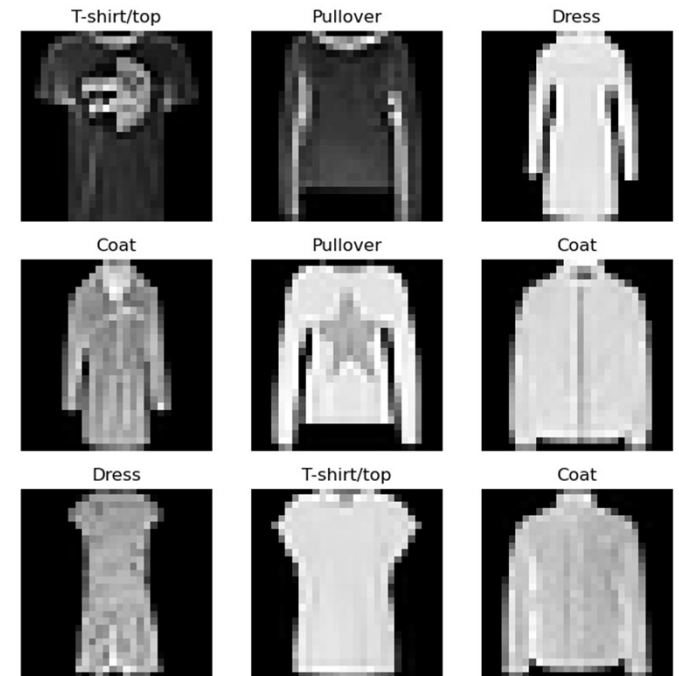


Pre-built datasets with torchvision

How can we use existing datasets?

- **Torchvision** provides popular datasets: MNIST, Cifar10, FashionMNIST, etc.
- Easy download and integration into PyTorch pipelines.

```
from torchvision.datasets import FashionMNIST
train_data = FashionMNIST(
    root="data",
    train="true",
    download=True
)
```





Preprocessing pipeline: Transforms

What are **Transforms**?

- Transformations applied when loading data as preprocessing steps.

Examples:

- **ToTensor()**: converts $[0, 255]$ pixel range $\rightarrow [0.0, 1.0]$, and reshapes $H \times W \times C \rightarrow C \times H \times W$.
- **Normalize(mean, std)**: normalize the image $\rightarrow$ speeds up convergence and stabilize training.

```
from torchvision import transforms
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

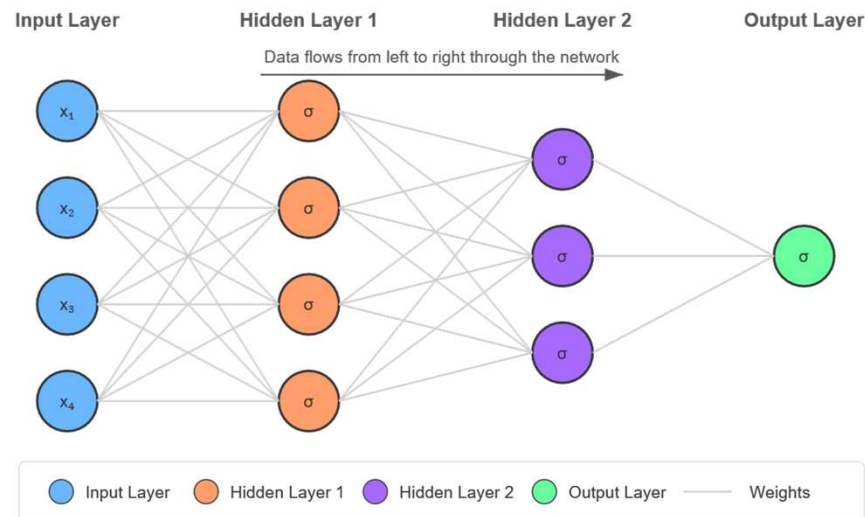


Multi-Layer Perceptron (MLP)

What is a **Multi-Layer Perceptron (MLP)**?

- Feedforward neural network with one or more hidden layers.
- Each layer: **linear transformation + activation function**.

Multi-Layer Perceptron (MLP) Architecture

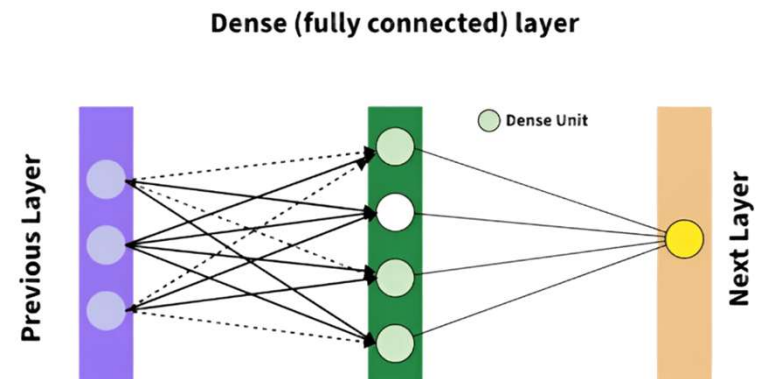




Linear (Fully Connected) Layer

- Computes: $y = Wx + b$.
- Maps input features to output features.
- Cannot model complex relationships.
- PyTorch:

```
import torch.nn as nn
linear = nn.Linear(in_features, out_features)
# x must have shape (batch_size, in_features)
y = linear(x)
```



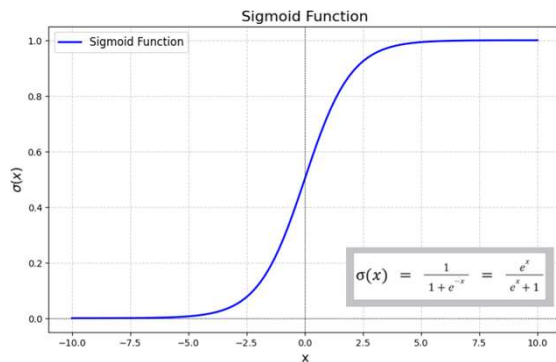


Activation Functions

- Introduce **non-linearity**, otherwise multiple linear layers collapse into one.
- Examples:

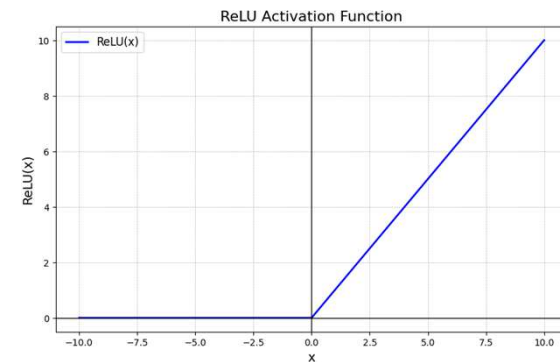
Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$

```
import torch.nn as nn
sigmoid = nn.Sigmoid()
y = sigmoid(y)
```



ReLU: $ReLU(x) = \max(0, x)$.

```
import torch.nn as nn
relu = nn.ReLU()
y = relu(y)
```





Loss Functions and Optimizers

Loss function:

- Measures difference between predictions and true labels.
- Guides optimization via gradients.
- Examples: **MSE** (regression), **Cross-Entropy** (classification), etc.

Optimizer:

- Updates model parameters to minimize loss.
- Examples: **SGD** (simple), **Adam** (faster convergence), etc.

```
import torch.nn as nn
criterion = nn.CrossEntropyLoss()
loss = criterion(predictions, targets)
```

```
from torch.optim import SGD
opt = SGD(model.parameters(), lr=0.01)
```



Training Loop & Eval Loop

- Training loop:

```
model.train()
for inputs, labels in train_loader:
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()
    ... # Compute metrics
```

- Eval loop:

```
model.eval()
with torch.no_grad():
    for inputs, labels in dev_loader:
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        ... # Compute metrics
```


Lab 5

Evolutionary Computation

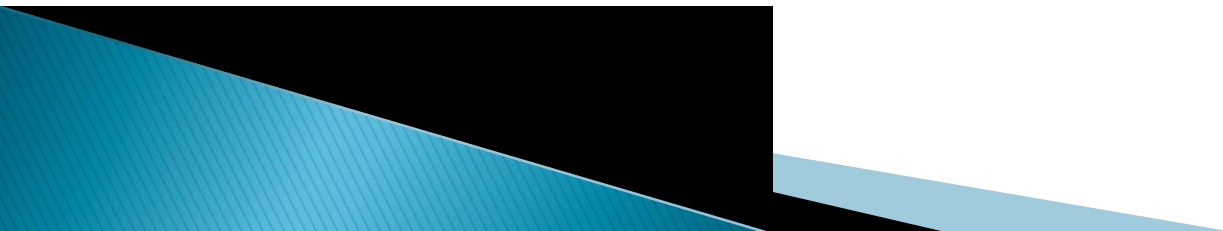
Genetic Algorithms

Software

- ▶ DEAP (Distributed Evolutionary Algorithms in Python)
 - A Python environment for Evolutionary Computation, including:
 - Genetic Algorithms
 - Genetic Programming
 - Particle Swarm Optimization
 - ... and more
- ▶ Homepage: <https://pypi.org/project/deap/>
- ▶ Documentation at: <https://deap.readthedocs.io/en/master/index.html>
- ▶ Install the most recent version by typing
 - `pip install git+https://github.com/DEAP/deap@master`

DEAP

A few examples, adapted from the examples one can find in the documentation, can be found on Elly in the archive `deap-examples_1.zip` (frequently updated, check if you have the latest version)



DEAP: setting up a GA

(refer to DEAP documentation for more details)

- ▶ Select the representation (**create** an Individual)
- ▶ Define an objective function (**create** a fitness function)
 - What the (**problem-dependent***) fitness function must do:
 - Decode (if necessary) the representation into a ‘working’ solution
 - Apply the solution to the problem at hand and compute its fitness.
- ▶ Create a Toolbox object and **register** the individual and the fitness function

* Since the fitness function is problem-dependent, one necessarily needs to write at least the fitness function: ‘closed’ packages (e.g. Weka Explorer) cannot be provided for algorithms.

DEAP: setting up a GA

- ▶ Specify if it is a minimization or maximization problem by setting the **weights** of the fitness function(s)
 - The fitness function is actually a weighted sum of functions: a negative weight means the function is to be minimized, a positive one means it is to be maximized
- ▶ **Register** a suitable mutation operator
- ▶ **Register** a suitable recombination (crossover) operator
- ▶ **Register** a selection strategy
- ▶ **Register** a population

DEAP: setting up a GA

- ▶ Instantiate the population
- ▶ Instantiate a *Statistics* object
- ▶ **Register** the statistics you want to collect
- ▶ Instantiate a *HallOfFame* object (a collection of the best individuals)
- ▶ Instantiate a Logbook object
- ▶ Choose an algorithm
- ▶ Run the algorithm, setting the parameters properly
 - If **verbose** is set to True, the statistics you have chosen will be displayed at the end of each generation, otherwise they are collected and returned in a Logbook object for subsequent use

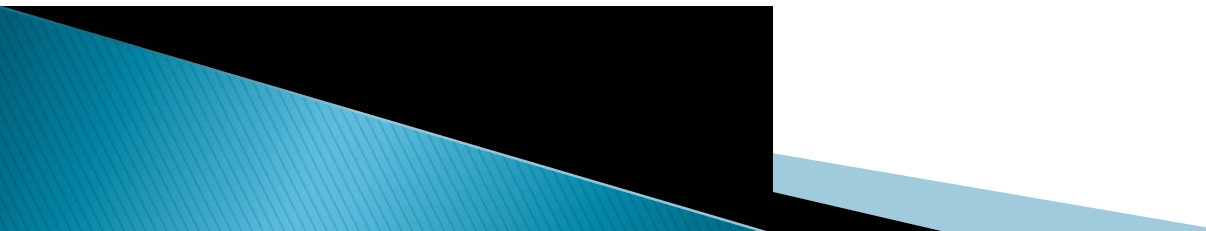
Exercise 1: Function Optimization

Examine the code of the Onemax example (simple-onemax.py) in the examples archive.

Based on it, set up a GA in DEAP which finds the minimum of the function

$$f(x,y,z) = (1.5 + \sin(z)) * (\sqrt{(20-x)^2 + (30-y)^2}) + 1$$

for $x, y, z \in [-250, 250]$



Exercise 1: Function Optimization

Write and compare the results of two versions that use:

- A list of 3 floating point numbers
 - Use Gaussian mutation with $\mu=0$ and $\sigma=0.2$ and two-point crossover)
- Lists of bits
 - Use a list of 90 integers in $[0,1]$ (bits) as a representation (see `simple-onemax.py`), assigning 30 bits to each variable to represent the interval $[-250, +250]$)
 - Use two-point crossover and flip-bit mutation

Exercise 1: Function Optimization

Notes:

- ▶ When using a list of N bits as a representation, the individual must be decoded within the fitness function

$$\text{Value}(i) = \text{Min} + i/(2^N - 1) * (\text{Max} - \text{Min})$$

where N is the number of bits used to represent a variable and i the unsigned integer represented by a string of bits ($i \in [0, 2^N - 1]$)

- ▶ Use the logbook and the graphic example provided by DEAP documentation to compare the convergence speed of the two algorithms, by plotting best and average fitness vs. generation number

Exercise 2: Pattern “guessing”

- ▶ Based on simple-onemax.py or onemax-numpy.py (a GA with a list of integers or a numpy.ndarray as representation) ‘guess’ the pattern stored in the file smiley.txt by finding an individual that matches it as closely as possible. It is actually the same problem as OneMax...

Using random guesses (brute-force approach) and without knowing the fitness of our solutions, it would take, on average, 2^{223} attempts to solve the problem!

(the pattern size is 16x14, so we have 2^{224} possible configurations)

```
000000000000000000
0000011111100000
0001100000001100
0010000000000100
0010011001100100
0100011001100010
0100000000000010
0100000000000010
0100000000000010
0010100000001010
0010011111100100
0001100000001100
0000011111100000
0000000000000000
```

Exercise 2: Pattern “guessing”

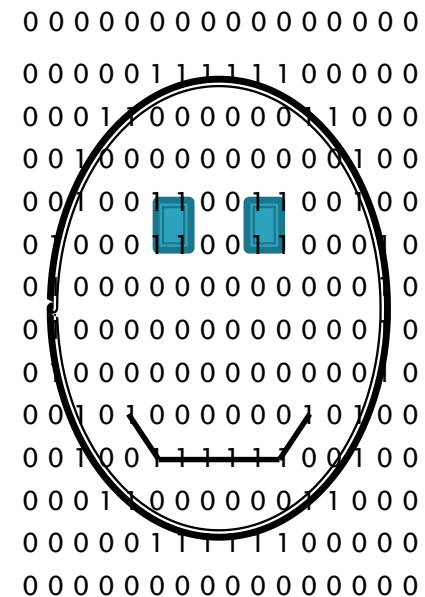
- Based on simple-onemax.py or onemax-numpy.py (a GA with a list of integers or a numpy.ndarray as representation) ‘guess’ the pattern stored in the file smiley.txt by finding an individual that matches it as closely as possible. It is actually the same problem as OneMax...

At 10^9 attempts/s, that would mean 2^{168} years!!!

(1 year $\sim 2^{25}$ s $10^9 \sim 2^{30}$)

A computer should have started before the Big Bang to find the solution now!

The smartest (but very specific) algorithm based on fitness would take at most 225 attempts. Can you figure it out?



```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 1 0 0 1 1 0 0 1 1 0 0 1 0 0 1 0 0 0
0 1 0 0 0 1 1 0 0 1 1 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 1 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0
0 0 1 0 0 1 1 1 1 1 1 0 0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0
0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Exercise 2: Pattern “guessing”

- ▶ Using patterns of increasing size (stored in files similar to smiley.txt) see how the number N_f of fitness function invocations needed to find the solution increases with pattern size. To do so, run the GA 5 times per pattern and compute the average of the five N_f values. (see also import-pattern-numpy.py)
- ▶ Do the same (repeat each experiment 5 times) using smiley.txt as the target but changing number of generations and population size. What is the best combination of the two parameters?

Exercise 3: N-queen problem (smart)

Problem: Place N queens on a $N \times N$ checkerboard such that none of them is checked

Fitness: number of queens which are checked

(smart) Representation: a permutation of values (0 to $N-1$)

NB Trivially, two queens cannot be aligned horizontally or vertically, so there must be one queen per row and one per column: the solution can thus be encoded as a sequence of integers corresponding to the position of a queen in each row, starting, for example, from the first row downwards.



Exercise 3: N-queen problem (smart)

Algorithm: genetic algorithm, tournament selection with tournament size = 3

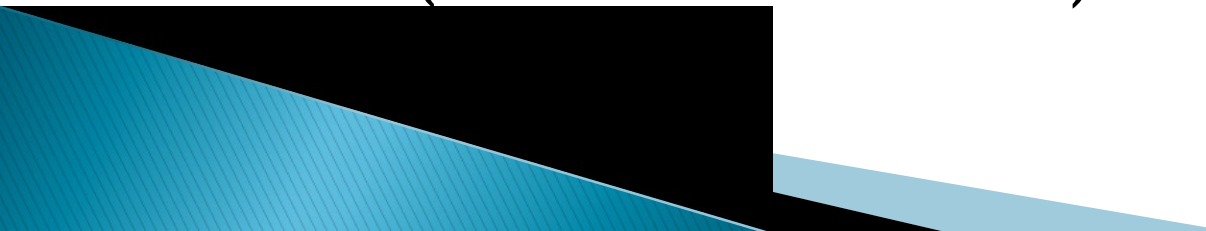
Operators: Mutation (shuffle permutation), PMX (crossover between permutations which guarantees that the result is still a permutation)

See `nqueens.py`

Parameters: population size = 300 , $P_c = 0.5$, $P_m = 0.2$

Number of Generations: 100

What is the largest N for which you can find a solution in most runs (success rate > 50%)?



Exercise 3: N-queen problem (dumb)

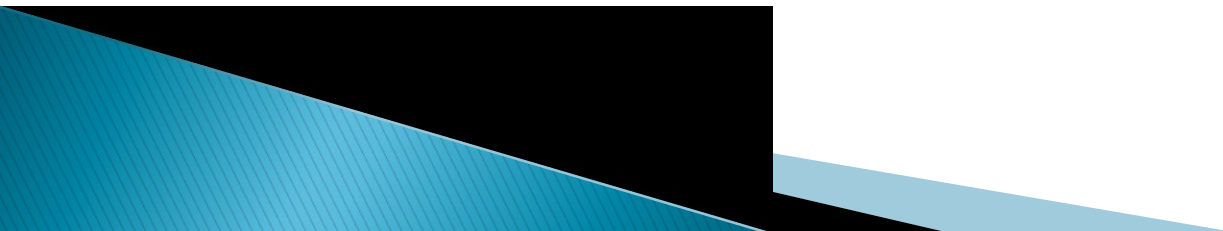
Using DEAP, write a GA that solves the N-queen problem using a $2*N$ integer representation (N pairs of coordinates between 0 and N-1), uniform-int mutation and single-point crossover, keeping all other parameters constant .

Such a representation does not take into account the fact that the solution must be a permutation.

Space search size (N=8) smart $8! = 40320$

dumb $2^{32} = 4294967296$

What is the largest N for which you can find a solution in most runs?



Exercise 4: Ant problem

Using DEAP, write a GA that finds the best solution to the ant game FOR A GIVEN BOARD CONFIGURATION.

Hints:

- ▶ Four possible moves (up, down, left, right) so each move can be encoded into two bits. Each game is composed of 20 moves. Thus, the solution can be represented by ...
- ▶ Fitness = score gained by the game represented by one individual.

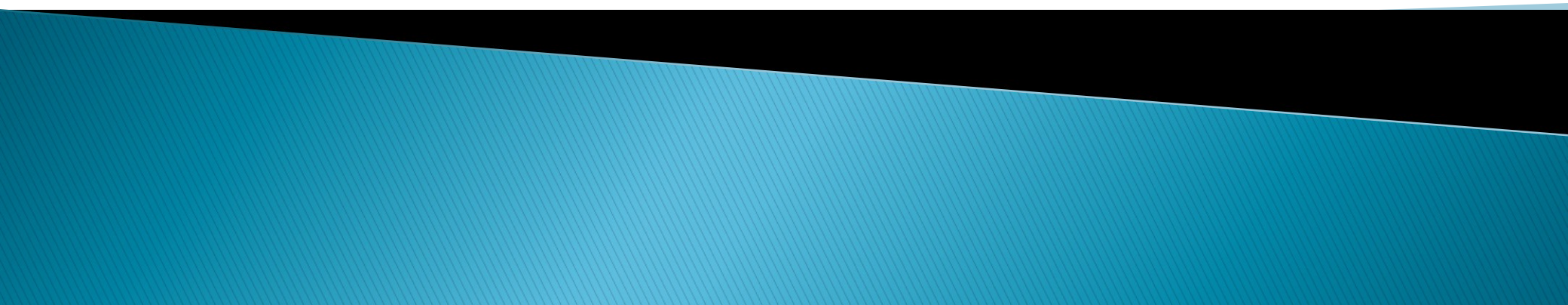
NB Of course, the solution found is a specific solution for a specific board configuration, NOT a game strategy!



Lab 6

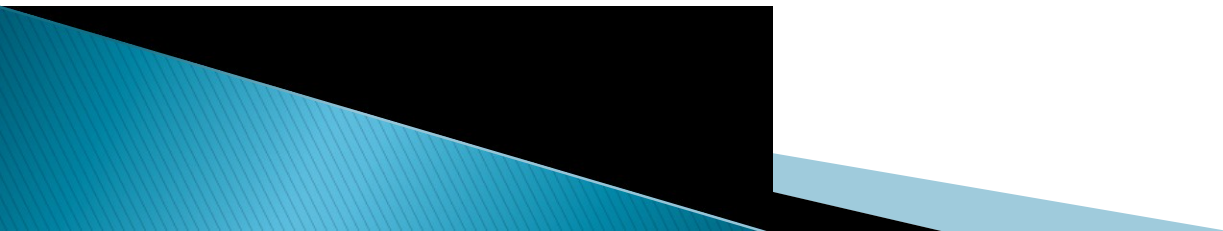
Evolutionary Computation

Genetic Programming and
Particle Swarm Optimization



Software

- ▶ Downloadable from Elly
(GPex0b.zip and deap-examples.zip)
 - Based on DEAP
 - Symbolic Regression using GP (exercise 0a)
 - Image denoising using GP (exercises 0b, 1)
 - Function optimization Particle Swarm optimization (ex 2)
- ▶ Follow the instructions in README.txt (GPex0b)



Exercise 0: symbolic regression using GP

Refer to the example `symbreg.py` provided by DEAP

Repeat the experiment defining a function set and a 2d function $f(x,y)$ at your will, which cannot be replicated exactly using the function set F .

Set the terminal set as $T = \{x, y, ERC_i\}$

ERC_i = Ephemeral Random Constant in $[1,10]$ $i=1,n$

Over a dense sampling of the domain, plot the original function and compare it to the approximated one.

Observe how results change when the GP parameters are modified.



Exercise 0b: GP-based image filtering

Given a greyscale image $I(x,y)$

create a noisy version

$$O(x,y) = I(x,y) + N(15)$$

with $N(15)$ uniform noise in $[-15, 15]$

Clip the values of $O(x,y)$ to 0 or 255 if they exceed the range above or below the extremes of the interval



Exercise 0b: GP-based image filtering

Find a sort of convolutional filter $f(N(x,y))$ such that $I'(x,y) = f * I(x,y)$ is as similar as possible to $I(x,y)$.

- ▶ $N(x,y)$ is a 3x3 neighborhood of (x,y)
- ▶ $*$ is the “quasi-convolution” operator (it is non-linear!)
 $f * I(x,y)$ means that the function f is to be applied to all pixel coordinates in $N(x,y)$ to produce an output image $G(x,y)$ where $G(x,y) = f(N(x,y)) \quad \forall (x,y)$

Exercise 0b: GP-based image filtering

Use terminal set

$$T = \{ O(x-1,y-1), O(x-1,y), O(x-1,y+1), O(x,y-1), O(x,y), O(x,y+1), O(x+1,y-1), O(x+1,y), O(x+1,y+1), ERC_i \ i=1,n \}$$

with ERC_i ephemeral random constants in $[0,K]$, with K and n arbitrary.

- ▶ Limit the co-domain of the functions in the function set F within $[0,255]$ (i.e., clip the values to 0 or 255 if they exceed the limits)
- ▶ Set tree maximum depth= N for the trees and suitable values for n , K and N .
- ▶ Denoise an image using the tree optimized on another image (or a different noise level) and compare the results.

Exercise 1: GP-based image filtering

Repeat the previous exercise but according to the following rules:

1. Take a low-quality greyscale image $I(x,y)$
2. Use a photo-editing application to improve its quality as you like
3. Name the resulting image $O(x,y)$
4. Use GP to find a transformation which, applied to $I(x,y)$, produces an image that is as similar as possible to $O(x,y)$.
5. Apply the GP tree to $I(x,y)$ to see the result.
6. Evaluate the generality of the solution by applying the same tree to another image which has not been used to construct the GP tree.

FEEL FREE TO CREATE NEW OPERATORS!!!

Exercise 2: PSO

Using Particle Swarm Optimization, find the maximum of the function

$$f(x,y,z) = (1 - \cos(2 * \pi / (1 + \exp(-(x-125)^2 - (y-1.27)^2)))) / (1 + z^2)$$

and analyze the results, setting the maximum number of iterations to 10000, according to the following rules:

1. Keeping 50 particles in the population, set $c1$, $c2$ in $[0,2]$ and w in $[0,1]$. Find good values for these three parameters.
2. Change the swarm size to 100, setting as values for w , $c1$ and $c2$ the values w' , $c1'$ and $c2'$ which have generated the best results in the previous step.
3. Verify whether other parameter sets exist which produce better results with a larger population.

Exercise 2: PSO

Remarks about PSO implementation

1. Limit the search space (in this exercise) within the intervals $x, y, z \in [-128, +128]$ considering the search space to be toroidal, i.e., consider -128 and 128 to be coincident. Then:

$128 + n$ ($0 < n < 256$) becomes $-128 + n$ and

$-128 - n$ ($0 < n < 256$) becomes $128 - n$

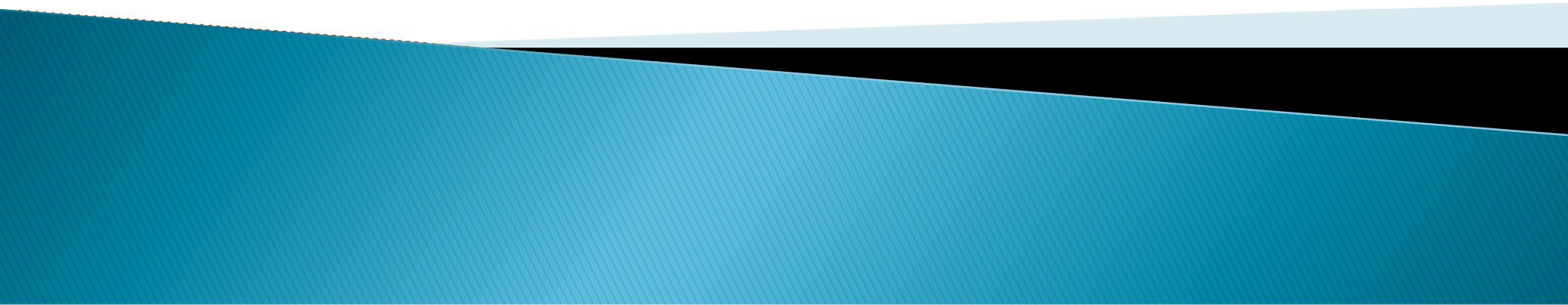
instead of clipping them to the extremes as it does now.

2. Constrain a particle's speed to an upper limit of a quarter of the search space width (in this case 64) to prevent excesses in speed value. If the limit is exceeded, speed can be set equal to its limit (it does already so).

Compare the two strategies (topologies) in point 1 and different speed limits in point 2

Lab X

The Ant Trail Problem



The Ant Trail Game

- ▶ Consider a game having the following rules:
 - An ant moves on a $N \times N$ -cell grid
 - N random cells are initially filled with 'food' (their value is +1); all other cells are empty and have value = 0
 - It is possible to control the motion of the ant in four directions: up, down, left, right (*choose between the world reference frame – the grid – and the ant's reference frame – its viewpoint*)
 - Initially the ant can occupy any empty cell, unless its position is specified otherwise. The cell value is set to -1 (*see the next slide*)
 - When the ant moves, it earns a score equal to the value of the cell into which it moves.

The Ant Trail Game

- When the ant occupies a cell, the cell value is decremented by 1
- If the ant exits the grid, the game ends: $N+2$ points are subtracted from its score. Equivalently, we can consider the cells immediately outside the grid to have a negative value of $-N-2$, besides being 'poisonous' for the ant that dies if it moves there
- The aim of the game is to maximize the score earned by the ant in $2N$ moves starting from a given cell
- Suppose the ant has a 'field of view' corresponding to a square neighborhood with side size $= 2m+1$ ('radius' $=m$) centered in the cell that the ant currently occupies. Therefore, the ant view can be described using the cells in its neighborhood as attributes and their corresponding content as attribute values

The Ant Trail Game

Write a function `total_score=ant_rec(N,m,filename,seed)` which allows one to 'move' the ant interactively, recording in `filename` all views (described by the ant's neighborhood) encountered during the game, along with the decision taken correspondingly, e.g., (m=1, 3x3 neighborhood) `0,0,1,0,-1,0,0,0,-1, right`. The random seed can be used to play different games using the same board initialization.

- ▶ *Use the data gathered in 1, 5, and 10 games as training sets to generate a "player program" using 1) a decision tree, 2) a multi-layer perceptron, and 3) a GP program. Set N=10 and m=1,2*
- ▶ Compare the results obtained in the 6 (x3 methods) different tests (each test is identified by a `[m, number_of_training_games]` pair), playing 5 test games (use always the same grid layout to allow for a fair comparison) moving the ant according to the decisions taken by the program

The Ant Trail Game

This means, given the two training data sets collected as described:

1. Train:

- A decision tree that leads to one of four decisions (move up, down, right, left)
- A multilayer perceptron with 9 (3x3 neighborhood) or 25 (5x5 neighborhood) inputs and one nominal output (classification problem, the output are 4 values, one for each direction)
- A Genetic Programming tree whose terminals are the 9 or 25 values of the cells in the ant's neighborhood and whose output OUT can be interpreted as:

up / ahead $-\text{inf} < \text{OUT} \leq -K$ [K is an arbitrary positive value]

down / backwards $-K < \text{OUT} \leq 0$

right $0 < \text{OUT} \leq K$

left $K < \text{OUT} \leq \text{inf}$

Use the four arithmetic operations (with protected division)
+ ERCs as the function set

The Ant Trail Game

This means, given the training data collected as described:

2. Create a set of 5 boards initialized with N food pieces and set a starting position for the ant
3. For each board, for $2 \times N$ moves, if the ant is 'alive':
 - Set the ant's score to 0
 - Collect the values of the cells in a 3×3 or 5×5 neighborhood of the ant's cell
 - Use them as the input to the trained model
 - Move the ant in the direction 'decided' by the model
 - Increase the score by the value of the new ant's cell and decrease the cell score by 1

The Ant Trail Game

Remarks

- ▶ If the ant is surrounded by cells having all the same value (in particular, null values) there is no preferential direction to move to: however, the program will always associate the same move to that state. So, in an empty region the ant tends to drift towards one of the sides. When it reaches it, it is surrounded by negative values (behind it and in front of it). If the tree has been built correctly, the ant should be able to change direction. However, exploration is not efficient.
- ▶ *Possible solution:* if there is no other information around the ant but the possible negative value of the cell visited in the previous step, generate a random move in any direction but the one from which the ant has come.

The Ant Trail Game

Remarks about the representation

- ▶ For each state, there are three other equivalent states (obtained by rotating the configuration and the decision by 90 degrees for 3 times): it is possible to increase the training set size exploiting those symmetries or to 'normalize' the representation such that equivalent states have one and only one representation (e.g., suppose the ant is always oriented in the North-South direction for the representation, then change the decision by rotating it by the same quantity as the actual orientation of the ant with respect to the North-South direction).
- ▶ Using a reference system oriented in the direction of the ant's motion (the ant always watches ahead of itself and 'annotates' what it sees from its viewpoint) the problem with symmetries is still there but the representation is more natural if one imagines to observe the world as the ant would see it.

The Ant Trail Game

More remarks

- ▶ *Information is incomplete* (the ant cannot see what is beyond the neighborhood): if the most relevant information is outside the neighborhood, playing the optimal move (based on full knowledge of the grid) when playing the 'training games' might generate moves that are inconsistent with the ant's *local* view.
- ▶ In general the optimal solution includes moves that keep the ant mainly close to the middle of the grid (it is easier to reach any other cell from the center of the grid). Because of this, the data gathered playing optimal games will not favor the development of «wall following» or «wall avoiding» behaviors, causing the generation of behaviors that tend to induce the ant to exit the grid.

The Ant Trail Game

Possible implementation

- ▶ Write the data collection `ant_rec` function
 - ▶ Write a function `show_board` that displays the board status
 - ▶ Write a function `ant_train` that trains a classifier using the data collected (using `sklearn`)
 - ▶ Write a function `ant_move` that plays a move given a board configuration and the model trained by `sklearn`
 - ▶ Anything else needed?
 - ▶ You may consider writing a class named `ant` and/or a class named `board`.
- 